ACADEMIC PROJECT MONITORING SYSTEM

PROJECT REPORT

Submitted in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications of A P J Abdul Kalam Technological University

Submitted by:

JESTIN GEORGE (SJC24MCA-2035)

MASTER OF COMPUTER APPLICATIONS

ST. JOSEPH'S COLLEGE OF ENGINEERING AND TECHNOLOGY, PALAI
(AUTONOMOUS)

CHOONDACHERRY P.O, KOTTAYAM

KERALA

MAY 2026

ST. JOSEPH' S COLLEGE OF ENGINEERING AND TECHNOLOGY, PALAI

(AUTONOMOUS)

(An ISO 9001: 2015 Certified College)

CHOONDACHERRY P.O, KOTTAYAM KERALA

CERTIFICATE

This is to certify that the project work entitled **ACADEMIC PROJECT MONITORING SYSTEM** submitted by **JESTIN GEORGE** student of **FOURTH** semester MCA at **ST. JOSEPH'S COLLEGE OF ENGINEERING AND TECHNOLOGY, P A L A I (AUTONOMOUS)** in partial fulfilment for the award of Master of Computer Applications is a Bonafide report of the PROJECT work carried out by him under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Ms. Binila Binoy	**Mr. Alex Jose**	**Dr. Rahul Shajan**
Assistant Professor	**Assistant Professor**	**Associate Professor**
(Project Guide)	**(Project Coordinator)**	**(Head of the Dept - CA)**

Submitted for the Viva-Voce Examination held on _____

Examiner 1:

Examiner 2:

DECLARATION

I **JESTIN GEORGE**, do hereby declare that the project titled **Academic Project Monitoring System** is a record of work carried out under the guidance of **Ms.Binila Binoy, Asst. Professor, Department of Computer Applications, SJCET, Palai** as per the requirement of the curriculum of Master of Computer Applications Programme of APJ Abdul Kalam Technological University,Thiruvananthapuram.Further, I also declare that this report has not been submitted, full or part thereof, in any University / Institution for the award of any Degree / Diploma.

Place: Choondacherry **JESTIN GEORGE**

Date : **(SJC24MCA-2035)**

ACKNOWLEDGEMENT

The success of any project depends largely on the encouragement and guidelines of many others. I would like to take this opportunity to express my gratitude to those people who have been instrumental in the successful completion of this project work. First and foremost, I give all glory, honour and praise to God Almighty who gave me wisdom and enabled me to complete the project successfully. I also express sincere thanks, from the bottom of my heart to my parents for their encouragement and support in all my endeavours and especially in this project. Words are inadequate to express my deep sense of gratitude to **Dr. V P Devassia, Principal, SJCET, Palai** for allowing me to utilize all the facilities of our college and also for his encouragement. I extend my sincere gratitude to **Dr. Rahul Shajan, Head of the Department of Computer Applications, SJCET, Palai** who has been a constant source of inspiration and without his tremendous help and support this project would not have been materialized.

I owe a particular debt of gratitude to my internal project guide, **Ms. Binila Binoy, Asst. Professor, Department of Computer Applications, SJCET, Palai** for all the necessary help and support that she has extended to me. Her valuable suggestions, corrections and the sincere efforts to accomplish my project even under a tight time schedule were crucial in the successful completion of this project. I extend my sincere thanks to all of our teachers and non-teaching staff of SJCET Palai for the knowledge they have imparted to me over the last two years. I would also like to express my appreciation to all my friends for their comments, help and support.

JESTIN GEORGE

(SJC24MCA-2035)

DEPARTMENT OF COMPUTER APPLICATIONS

Our Vision

"To emerge as a center of excellence in the field of computer education with distinct identity and quality in all areas of its activities and develop a new generation of computer professionals with proper leadership, commitment and moral values."

Our Mission

1. Provide quality education in Computer Applications and bridge the gap between the academia and industry.
2. Promoting innovation research and leadership in areas relevant to the socio-economic progress of the country.
3. Develop intellectual curiosity and a commitment to lifelong learning in students, with societal and environmental concerns.

PEO'S (Program Educational Objectives)

1. MCA Graduates will be able to progress career productively in software industry, academia, research, entrepreneurship pursuits, government, consulting firms and other IT enabled services.
2. MCA Graduates will be able to achieve peer-recognition as an individual or in a team by adopting ethics and professionalism and communicate effectively to excel well in crisis and inter-disciplinary teams.
3. MCA Graduates will be able to continue life-long professional development in computing and in management that contributes in self and societal growth.

COURSE OUTCOMES

MAIN PROJECT

SUBJECT CODE – 24SJMCA246

Co No	*CO*	*Blooms Category*
CO1	Identify a real-life project which is useful to society/ industry	Level 2: Understand
CO2	Interact with people to identify the project requirements	Level 3: Apply
CO3	Apply suitable development methodology for the development of the product / project	Level 3: Apply
CO4	Analyze and design a software product / project	Level 4: Analyze
CO5	Test the modules at various stages of project development	Level 5: Evaluate
CO6	Build and integrate different software modules	Level 6: Create
CO7	Document and deploy the product / project	Level 3: Apply

SYNOPSIS

The management of academic projects in higher educational institutions is a complex, multi-tiered process involving coordinators, faculty guides, and students. Traditional approaches are heavily reliant on manual tracking and decentralized tools, resulting in fragmented communication, delayed approvals, inconsistent evaluations, and lack of transparency — ultimately affecting project quality and academic outcomes.

The Academic Project Monitoring System is a modern, role-based web application designed to streamline the entire lifecycle of student academic projects — from topic submission to final evaluation and archival — using secure, efficient, and user-friendly technology. The system integrates project workflows, milestone management, secure document versioning, and analytical reporting into a single centralized platform.

The platform provides dedicated dashboards for Students, Guides (faculty supervisors), and Coordinators (HODs/Department heads). Students can submit project topics, track progress, upload documents, and communicate with guides. Guides can approve topics, define milestones, provide inline feedback, and monitor group performance. Coordinators manage guide allocation, set deadlines, configure batches, and access department-level analytics.

The system is developed using modern technologies, including a React-based frontend, a Node.js/Express and FastAPI backend, and a PostgreSQL relational database. Security is ensured through mechanisms like JWT-based authentication and encrypted data transmission. The platform supports real-time communication, Agile-style task boards, rubric-based evaluation, and plagiarism check integration.

The Academic Project Monitoring System aims to improve project quality, enhance guide-student communication, and support data-driven decision-making in educational institutions. By combining automation, structured workflows, and a centralized digital platform, the system transforms traditional project supervision into an efficient and transparent process.

CONTENTS

INTRODUCTION

1. INTRODUCTION

The management of academic projects in higher educational institutions has always been a complex, multi-stakeholder process. Students must submit topics, track progress, upload deliverables, and receive timely feedback, while guides must supervise multiple groups, define milestones, provide inline comments, and evaluate performance consistently. Coordinators face the challenge of allocating guides fairly, monitoring batch progress, and ensuring compliance with academic deadlines. The Academic Project Monitoring System is a comprehensive, full-stack web application designed to digitize and streamline the entire lifecycle of student academic projects. It bridges the critical gap between students, faculty guides, and department coordinators by consolidating project workflows into a centralized, role-based digital platform. The system fosters transparency, improves coordination among stakeholders, and ensures consistency in evaluation and progress tracking.

The platform is built with a React frontend providing an intuitive, modern interface for all user roles. The backend is developed using Node.js with Express for API management and FastAPI for high-performance services, with PostgreSQL as the relational database. Authentication is secured via JWT-based mechanisms, and all data transmissions are protected through encrypted channels.

The Academic Project Monitoring System defines distinct roles with specialized dashboards and responsibilities: Student, Guide (Faculty Supervisor), and Coordinator (HOD/Department Head). Each role interacts with the system through purpose-built interfaces that expose only relevant functionality, ensuring data privacy and operational clarity. The system supports the complete project lifecycle from group formation and topic submission through milestone tracking, document versioning, rubric-based evaluation, and final project archival.

A standout feature of the system is its integrated Analytics and Reporting module, which provides personal progress dashboards for students, group-level reports for guides, and department-wide oversight for coordinators. Risk alerts help coordinators identify at-risk projects based on inactivity or missed deadlines. The system also supports Agile-style task boards (Kanban view) with sprint-based progress tracking, for students, combining regular class periods with AI-recommended study blocks tailored to each student's academic weaknesses.

This project report presents a comprehensive overview of the Academic Project Monitoring

System, covering its objectives, system analysis, design architecture, database schema, technical implementation, security policies, and future scope.

ABOUT THE ORGANIZATION

2. ABOUT THE ORGANIZATION

The establishment of St. Joseph's College of Engineering was the fulfilment of a long cherished dream of providing facilities for higher education to the people of the diocese and surrounding regions. The main objective is to develop a college with a distinct identity and character, where education and training are imparted in a truly Christian environment conducive to fostering Christian values such as faith in God, love for their fellow men and devotion to the motherland. Every facility is provided in the campus to create an environment fully conducive to realizing this objective.

Discipline, hard work, positive thinking, commitment to excellence and abiding faith in the Almighty are the guiding principles that propel the college to its vision of emerging as a Centre of Excellence in technical education in the country. Value systems such as eco friendliness, quality consciousness and work ethics are also being instilled through the special work culture and campus life existing in the college. The college aims to provide an education that WORKS! – An education that helps the students in ensuring a challenging and satisfying career after the course.

SYSTEM ANALYSIS

3. SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

In most educational institutions, the management of academic projects is largely manual, fragmented, and inefficient. Students form groups and submit project proposals on paper or via Google Forms, and guides track their progress through physical registers or personal spreadsheets, making data access and analysis difficult.

Guide allocation is performed by the Department Head using Excel spreadsheets, making it difficult to visualize faculty workload balance. Document submissions such as SRS reports, design documents, and final reports are handled via email or physical printouts. Guides must download files, annotate externally, and email feedback back to students, leading to version confusion and lost feedback.

Furthermore, the absence of a centralized system results in delayed approvals, inconsistent evaluations, poor communication, increased paperwork, and lack of transparency. Coordinators struggle to obtain real-time visibility into project progress, guide performance, and departmental compliance — ultimately affecting the overall quality of academic project outcomes.

3.2 PROPOSED SYSTEM

The proposed system, the Academic Project Monitoring System, is a role-based web platform designed to address these challenges by consolidating all project-related activities into a single, secure digital environment.

The system provides a centralized digital platform that integrates project lifecycle management, document workflows, real-time communication, and departmental analytics. It collects and stores all project-related data — topics, milestones, tasks, documents, feedback, and evaluations — in a structured PostgreSQL database.

Role-specific dashboards are provided for students, guides, and coordinators. Automated notifications alert users about deadlines, pending approvals, and feedback updates, enabling timely and effective action across all stakeholders.

Students can submit project topics, manage Agile-style task boards, upload versioned documents, and schedule meetings with guides. Guides can approve topics, define milestones, review documents with inline feedback, and rate student performance. Coordinators oversee entire system, ensuring efficient data management and smooth operation.

By implementing the Academic Project Monitoring System, the institution improves transparency, enhances guide-student coordination, enables consistent rubric-based evaluation, and supports data-driven academic oversight across all departments.

3.3 SOFTWARE REQUIREMENT SPECIFICATION(SRS)

3.3.1 Problem to Be Solved

The academic project management process in educational institutions often suffers from inefficiencies due to manual operations and lack of system integration. Activities such as topic submission, guide allocation, document review, and milestone tracking are handled through separate tools or manual methods, leading to errors, delays, and lack of transparency.

There is no centralized system to monitor project progress in real time or to identify at-risk projects early. Project supervision is unstructured, and there is no proper mechanism to track guide feedback, document versions, or milestone completion status.

The Academic Project Monitoring System addresses these challenges by providing a unified digital platform that integrates project data, supervision workflows, and analytical reporting. It enables real-time monitoring, improves coordination among all stakeholders, and supports proactive academic decision-making.

3.3.2 Customer Requirements

Users of the Academic Project Monitoring System require a simple, reliable, and intuitive platform to manage all aspects of academic project supervision efficiently.

- **Students** need tools for topic submission, task management, document upload, and feedback review
- **Guide** require tools to monitor student progress and provide structured guidance
- **Administrators** need centralized control over academic data and system operations

All users expect secure access, real-time updates, and a system that ensures transparency, accuracy, and ease of use.

3.3.3 What the developers need to know

Developers must understand the academic project lifecycle, including topic submission, guide allocation, milestone management, document versioning, evaluation workflows, and analytics reporting.

The system should:

- Implement role-based access control
- Support real-time data updates
- Integrate AI models for risk prediction
- Ensure efficient database handling
- Provide notification and alert mechanisms

Scalability, performance, and security must be considered during development.

3.3.4 Business Requirements

The Academic Project Monitoring System is designed to improve project quality and supervision efficiency by providing a centralized platform for managing all aspects of academic project workflows.

The system should:

- Reduce academic failures and dropout rates
- Improve institutional efficiency
- Support scalability for multiple departments or institutions
- Enable future enhancements such as advanced analytics and integrations

3.3.5 User Requirements

Users should be able to access the system based on their respective roles. Students should be able to submit project topics, manage Agile task boards, upload versioned documents, view feedback, and schedule meetings with guides. Guides should be able to approve topics, define milestones, review documents with inline comments, rate student performance, and export

progress reports. Coordinators should have the ability to manage batches, allocate guides, set deadlines, monitor departmental progress, and access analytics dashboards. The system must provide secure login functionality, real-time notifications, easy navigation, and a responsive interface that works seamlessly across different devices.

3.3.6 Functional Requirements

The system should support secure authentication and role-based access control to ensure that users can only access features relevant to their roles. It should manage all project-related data including topics, milestones, tasks, documents, and evaluations efficiently. The system must provide real-time notifications to alert users about deadlines, approvals, and feedback updates. It should enable meeting scheduling between students and guides and facilitate communication through in-app chat. Additionally, the system should maintain activity logs and audit trails for transparency and future reference.

3.3.7System Requirements Hardware Requirements (H/W)

The system requires a cloud-based or on-premise server to handle data processing and user requests efficiently. Administrators and Guide should use laptops or desktops with at least an Intel Core i5 processor and 8GB RAM for smooth operation. Students can access the system using mobile devices or personal computers with a minimum of 3GB RAM. A stable internet connection is essential for real-time updates and seamless system functionality.

Software Requirements (S/W)

The system is developed using React for the frontend, while the backend is built using Node.js with Express and FastAPI frameworks. PostgreSQL is used as the primary relational database for efficient and structured data management. Development tools include VS Code, Postman, and Git for coding, testing, and version control. The system is compatible with modern web browsers such as Google Chrome, Mozilla Firefox, and Microsoft Edge.

Edge.

3.3.7 Non-Functional Requirements

The system must ensure high performance and responsiveness to handle multiple users efficiently. It should be scalable to accommodate increasing numbers of users and data. Strong security and data protection mechanisms must be implemented to safeguard sensitive information. The system should maintain high availability and reliability to ensure

uninterrupted service. Additionally, it must provide a user-friendly interface that is easy to understand and operate..

3.3.8 Unstated Requirements

Users may expect additional features such as mobile application support, advanced dashboards for better data visualization, AI-based recommendations for academic improvement, and integration with other academic systems to enhance overall functionality and user experience.

3.3.9 Statutory & Regulatory Requirements

The system must comply with data protection and privacy policies to ensure the security of user information. It should also adhere to institutional academic regulations and standards for handling student data. Proper data handling practices must be followed to maintain confidentiality, integrity, and secure access to system data.

3.3.10 Quality Attributes

The system should be reliable, ensuring consistent performance without failures. It must maintain strong security to protect sensitive data. Scalability is important to support future growth, while performance efficiency ensures quick response times. The system should be user-friendly for ease of use and maintainable to allow future updates and enhancements.

3.3.11 Constraints

The system depends on internet connectivity, which may affect real-time operations in case of network issues. Budget and resource limitations may restrict certain advanced features. Users may initially face challenges adapting to the new system. Additionally, dependency on external APIs and services may impact system functionality if those services change or become unavailable.

3.4 FEASIBILITY ANALYSIS

3.4.1 Operational Feasibility

The Academic Project Monitoring System is designed to improve project supervision processes by integrating students, guides, and coordinators into a single platform. It reduces manual effort, enhances coordination, and ensures transparency in project activities such as milestone tracking, document review, and evaluation. Students can easily manage their project workflows,

guides can monitor group progress and provide timely feedback, and coordinators can oversee departmental operations efficiently. Since the system improves efficiency, accuracy, and user experience, it is operationally feasible for implementation in educational institutions.

3.4.2 Technical Feasibility

The Academic Project Monitoring System can be developed using modern and widely available technologies such as React for the frontend and Node.js/Express with FastAPI for the backend. PostgreSQL ensures secure and efficient relational data management. The integration of plagiarism check services and notification systems is supported by widely available APIs, making implementation practical. Cloud platforms can be used for deployment to ensure scalability, performance, and security. Considering the availability of tools, technologies, and technical expertise, the system is technically feasible.

3.4.3 Economic Feasibility

The development cost of the Academic Project Monitoring System is minimized by utilizing open-source technologies and scalable cloud infrastructure. The system reduces manual workload for guides and coordinators, saving time and operational costs. Maintenance expenses remain manageable due to efficient system design and modern development practices. Since the platform provides long-term benefits such as improved project quality, better guide-student coordination, and enhanced departmental efficiency, it is economically feasible.

3.5 DATA FLOW DIAGRAM (DFD)

A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It can be manual, automated, or a combination of both. It shows how data enters and leaves the system, what changes the information, and where data is stored. The objective of a DFD is to show the scope and boundaries of a system as a whole. It may be used as a communication tool between a system analyst and any person who plays a part in the order that acts as a starting point for redesigning a system. The DFD is also called as a data flow graph or bubble chart.

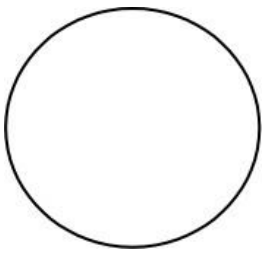
The following observations about DFDs are essential:

a. All names should be unique. This makes it easier to refer to elements in the DFD.
b. Remember that DFD is not a flow chart. Arrows is a flow chart that represents the order of events; arrows in DFD represent flowing data. A DFD does not involve any order of events.
c. Suppress logical decisions. If we ever have the urge to draw a diamond-shaped box in a DFD, suppress that urge! A diamond-shaped box is used in flow charts to represent decision points with multiple existing paths of which the only one is taken. This implies an ordering of events, which makes no sense in a DFD.
d. Do not become bogged down with details. Defer error conditions and error handling until the end of the analysis.

Data Flow Diagram Symbols

External entity: An outside system that sends or receives data, communicating with the system being diagrammed. They are the sources and destinations of information entering or leaving the system. They might be an outside organization or person, a computer system or a business system. They are also known as terminators, sources and sinks or actors. They are typically drawn on the edges of the diagram.

Process: Input to output transformation in a system takes place because of process function. The symbols of a process are rectangular with rounded corners, oval, rectangle or a circle. The process is named a short sentence, in one word or a phrase to express its essence.



Data Flow: Data flow describes the information transferring between different parts of the

systems. The arrow symbol is the symbol of data flow. A relatable name should be given to the flow to determine the information which is being moved. Data flow also represents material along with information that is being moved. Material shifts are modeled in systems that are not merely informative. A given flow should only transfer a single type of information. The direction of flow is represented by the arrow which can also be bi-directional.

Data Store: Also known as warehouse. The data is stored in the warehouse for later use. The warehouse is simply not restricted to being a data file rather it can be anything like a folder with documents, an optical disc, a filing cabinet. The data warehouse can be viewed independent of its implementation. When the data flows from the warehouse it is considered as data reading and when data flows to the warehouse it is called data entry or data updation.

Levels in Data Flow Diagram

The DFD may be used to perform a system or software at any level of abstraction. DFDs may be partitioned into levels that represent increasing information flow and functional detail. Levels in DFD are numbered 0, 1, 2 or beyond.

Level - 0 DFD

It is also known as fundamental system model, or context diagram represents the entire software requirement as a single bubble with input and output data denoted by incoming and outgoing arrows.

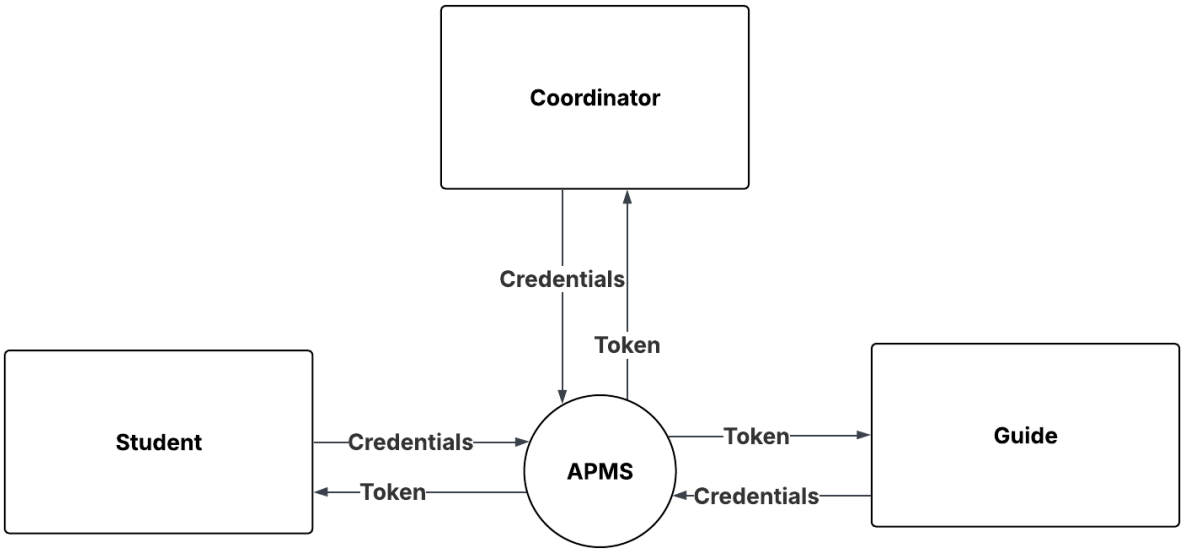
Level - 1 DFD

In level 1 DFD, a context diagram is decomposed into multiple bubbles/processes. In this level, we highlight the main objectives of the system and breakdown the high-level process of 0-level DFD into subprocesses.

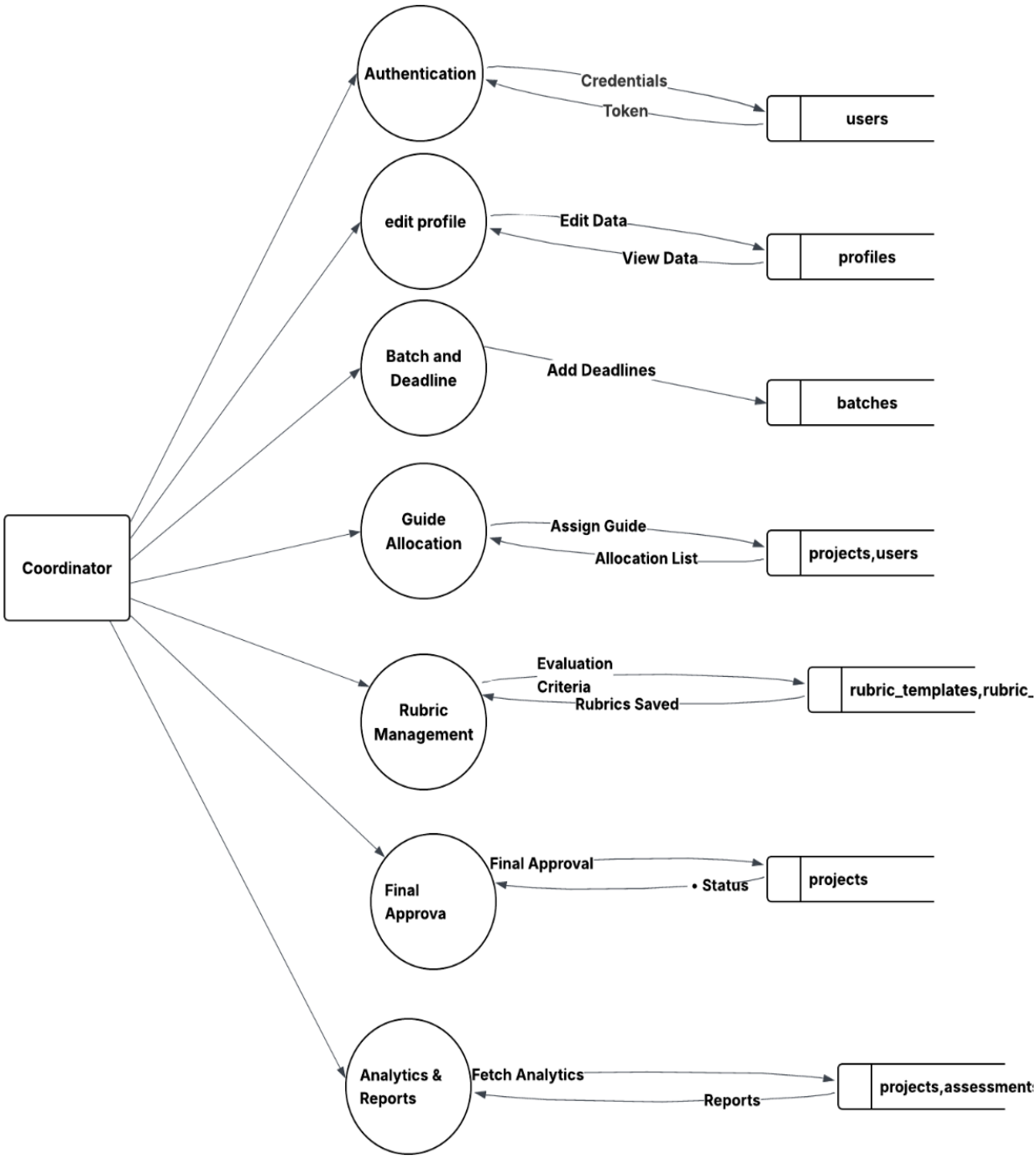
Level - 2 DFD

Level 2 DFD goes one process deeper into parts of level 1 DFD. It can be used to project or record specific/necessary detail about the system's functioning.

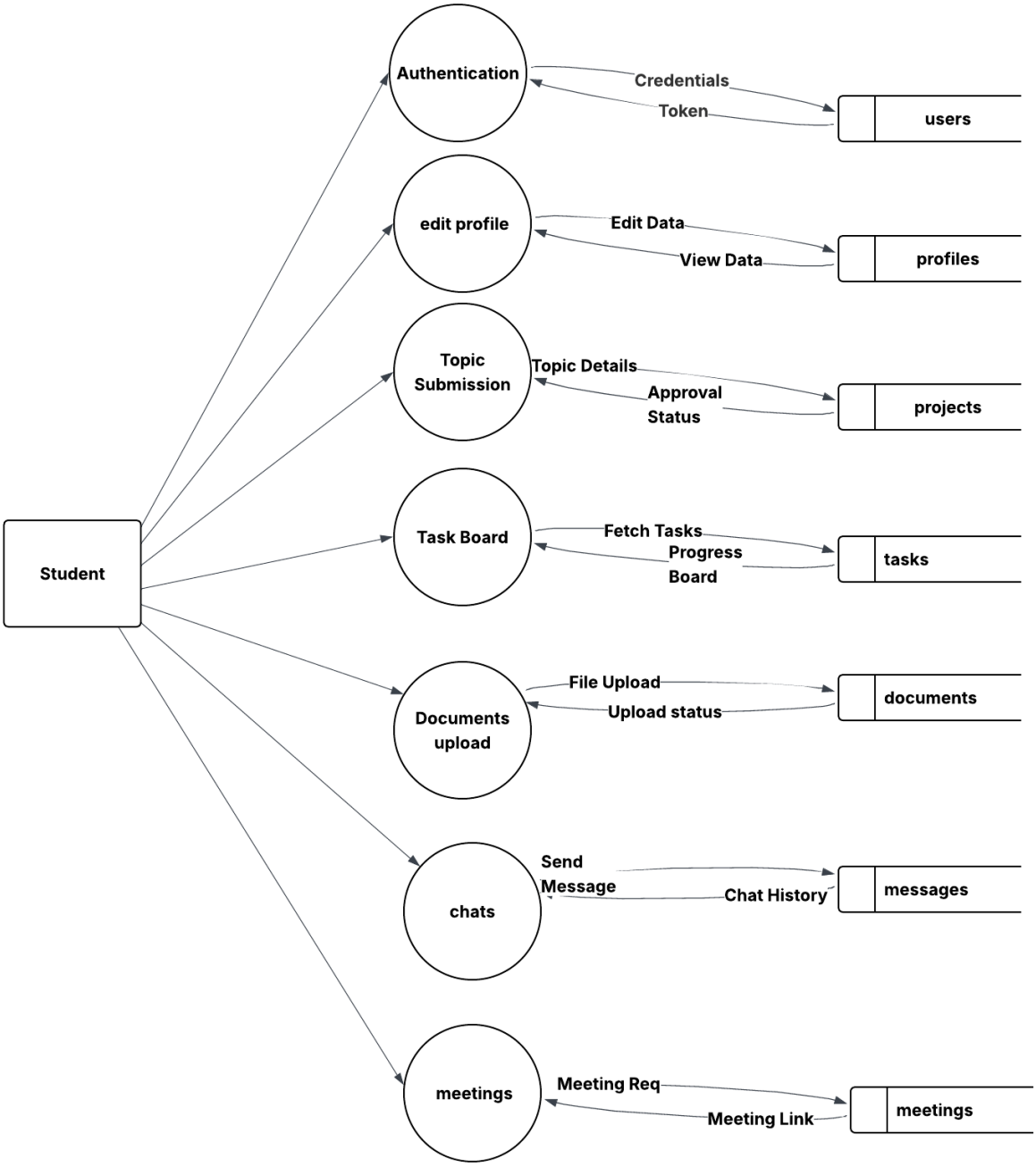
LEVEL 0 DFD


Coordinator
Credentials
Token
Student
Credentials
Token
APMS
Token
Credentials
Guide

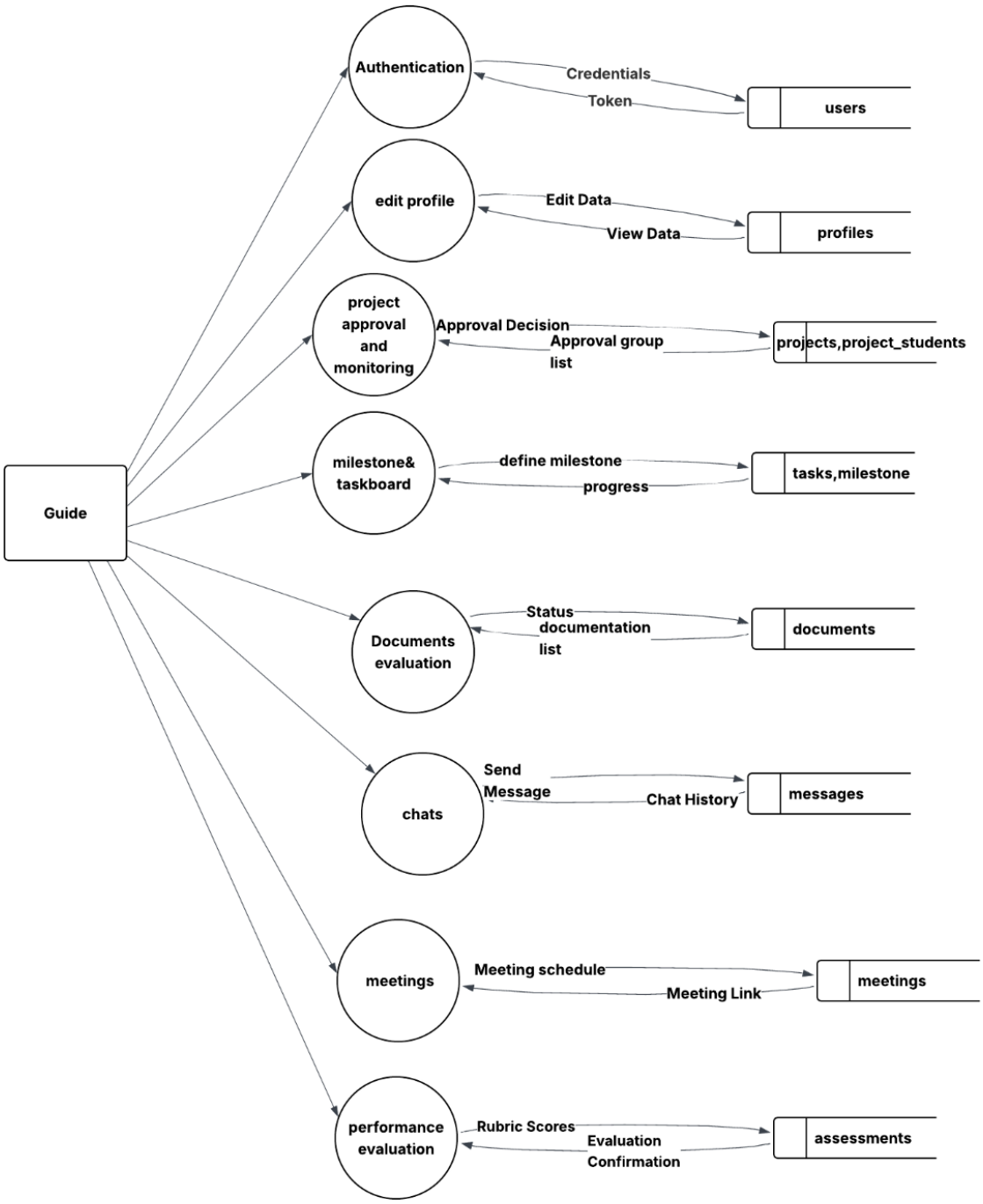
LEVEL 1 – Coordinator



LEVEL 1 – STUDENT



LEVEL 1 –Guide



SYSTEM DESIGN

4. SYSTEM DESIGN

System design acts as the blueprint for how the Academic Project Monitoring System functions, ensuring efficiency, scalability, and seamless integration of its components. It defines the overall structure of the system, including user inputs, data processing workflows, database architecture, and output presentation.

The platform provides structured and user-friendly interfaces for students, guides, and coordinators to manage academic project activities effectively. The database, implemented using PostgreSQL, stores user profiles, project data, tasks, documents, deadlines, meetings, evaluations, and messages, enabling fast retrieval and real-time updates.

The system's processing logic supports project lifecycle management, document version control, milestone tracking, rubric-based evaluation, and analytical reporting. Outputs are presented through role-specific dashboards, alerts, and exportable reports, allowing users to monitor project progress and take necessary actions. This ensures a proactive, organized, and transparent academic project supervision system.

4.1 INPUT DESIGN

Input design focuses on how data is entered into the Academic Project Monitoring System to ensure accuracy, efficiency, and ease of use. The platform provides structured input forms for students, guides, and coordinators.

Coordinators can input and manage data such as batch configurations, guide allocations, deadlines, and rubric criteria through well-designed forms. Guides provide inputs by reviewing submitted topics, defining milestones, annotating documents with inline feedback, and scoring student groups. Students interact with the system by submitting project topics, creating and updating tasks on the Kanban board, uploading versioned documents, requesting meetings, and submitting self-evaluations.

The system incorporates strong validation techniques, including mandatory fields, format validation for email and academic identifiers, dropdown selections for structured data, and real-time error messages. These validation mechanisms ensure that incorrect or incomplete data is minimized, enabling accurate AI analysis and system processing.

4.2 OUTPUT DESIGN

Output design defines how information is presented to users in a clear, organized, and meaningful manner. The Academic Project Monitoring System provides real-time outputs related to project progress, document review status, evaluation results, and departmental analytics.

Students can view their project progress, task completion status, document feedback, meeting history, and evaluation scores through intuitive dashboards. Guides receive detailed insights into group performance, pending document reviews, and milestone completion status. Coordinators can access comprehensive reports, including department-level project analytics, guide workload summaries, risk alerts for delayed projects, and overall batch compliance metrics.

The system generates structured outputs such as project progress reports, rubric-based evaluation summaries, and exportable group reports in PDF or Excel format, ensuring clarity and usability. Automated notifications and alerts keep users informed about upcoming deadlines, document approvals, feedback updates, and meeting requests, enabling timely actions and better academic oversight.

4.3 TABLE DESIGN

Database design is a critical process in the development of computer-based systems that store and manage data. It involves defining the structure and organization of a database to ensure data accuracy, integrity, security, and efficient retrieval. Effective database design is essential for applications ranging from small websites to large-scale enterprise systems. A primary key is the column or columns that contain values that uniquely identify each row in a table. A database table must have a primary key for option to insert, update, restore, or delete data from a database table.

Normalization: The term normalization refers to the way data item is grouped together into record structures. In other words, normalization is a technique of separating redundant fields and breaking up large tables into smaller ones. After the conceptual level, the next level of process of database design to organize the database structure into a good shape called Normalization. Normalization is adopted to overcome drawbacks like repetition of data, loss of information, Inconsistence. In our design, all tables have been normalized up to the third normal

form.

The different normal forms applied during the database design are given below.

- First Normal Form (1NF)
- Second Normal Form (2NF)
- Third Normal Form (3NF)
- Boyce-Codd Normal Form (BCNF)

FIRST NORMAL FORM

A relation is said to be in 1NF if it satisfies the constraints that it contains primary key.

SECOND NORMAL FORM

A relation is said to be in 2NF if and only if it satisfies all 1NF conditions for the primary key and every non-primary key attribute of the relation is fully dependent on its primary key alone. If a non-key attribute is not dependent on the key, it should be removed from the relation and placed as a separate relation i.e. the fields of the table in 2NF are all related to primary key.

THIRD NORMAL FORM

A relation is said to be in 3NF if only if only it is in 2NF and more over non-key attributes of the relation should not depend on other non-key attributes. The data in the system has to be stored and retrieved from database.

BOYCE-CODD NORMAL FORM

Boyce-Codd Normal Form is a advanced version of Third Normal Form. Here we have some additional rules than Third Normal Form. The basic condition for any relation to be in BCNF is that:

1. Table must be in Third Normal Form.
2. In relation X->Y, X must be a superkey in a relation.

The database schema will be structured to efficiently store and retrieve accident-related data. Key tables will include:

1. Table: Users

Description: This table stores all system users including students, guides, and coordinators.

Sl.No	Field Name	Data Type	Size	Constraints
1	uid	VARCHAR	255	Primary Key
2	email	VARCHAR	100	Not Null
3	role	VARCHAR	100	Not Null
4	auth_provider	VARCHAR	20	
5	oauth_id	VARCHAR	255	
6	Is_active	BOOLEAN	15	DEFAULT TRUE
7	created_at	TIMESTAMPTZ		DEFAULT NOW()

2. Table: Profiles

Description: Stores extended profile information for each user.

Sl.No	Field Name	Data Type	Size	Constraints
1	pid	VARCHAR	55	Primary Key
2	uid	VARCHAR	50	Foreign key
3	password_hash	VARCHAR	255	Not Null
4	first_name	VARCHAR	50	Not Null
5	Last_name	VARCHAR	50	
6	Profile_img	VARCHAR	500	

3. Table: Departments

Description: Stores department information and coordinator assignments.

Sl.No	Field Name	Data Type	Size	Constraints
1	deptid	VARCHAR	55	Primary Key
2	name	VARCHAR	50	Not null
3	Coordinator_id	VARCHAR	100	Foreign Key
4	Created_at	TIMESTAMPTZ	100	DEFAULT NOW()

4. Table: Batches

Description: Stores academic batch configurations per department.

Sl.No	Field Name	Data Type	Size	Constraints
1	batchid	VARCHAR	55	Primary Key
2	name	VARCHAR	50	Not null
3	Dept_id	VARCHAR	100	Foreign Key
4	Is_active	Boolean		Default True

5. Table: Groups

Description: Stores student project groups and their assigned guides.

Sl.No	Field Name	Data Type	Size	Constraints
1	groupid	VARCHAR	55	Primary Key
2	name	VARCHAR	100	Not null
3	batch_id	VARCHAR		Foreign Key
4	Guide_id	VARCHAR		Foreign key

6. Table: Group Members

Description: Maps students to their respective project groups.

Sl.No	Field Name	Data Type	Size	Constraints
1	memberid	VARCHAR	55	Primary Key
2	Group_id	VARCHAR		Foreign Key
2	Student_id	VARCHAR		Foreign Key
3	Is_leader	Boolean		DEFAULT FALSE

7. Table: Projects

Description: Stores all academic project details submitted by groups.

Sl.No	Field Name	Data Type	Size	Constraints
1	Project_id	VARCHAR	55	Primary Key
2	Group_id	VARCHAR		Foreign Key
3	Title	INTEGER		Not Null
4	domain	INTEGER		Not Null
5	description	TEXT		NOT NULL
6	status	VARCHAR		
7	Repo_link	VARCHAR	500	

8. Table: Tasks

Description: Stores Agile tasks created within a project.

Sl.No	Field Name	Data Type	Size	Constraints
1	Task_id	VARCHAR	55	Primary Key
2	Project_id	VARCHAR		Foreign Key
3	title	VARCHAR	150	NOT NULL
4	status	VARCHAR		NOT NULL

9. Table: Documents

Description: Stores uploaded project documents with review status.

Sl.No	Field Name	Data Type	Size	Constraints
1	Doc_id	VARCHAR	55	Primary Key
2	Project_id	VARCHAR		Foreign Key

Sl.No	Field Name	Data Type	Size	Constraints
3	File path	VARCHAR	500	Not null
4	type	VARCHAR	50	
5	name	TEXT		

10. Table: Deadlines

Description: Stores deadline configurations set by coordinators for each batch.

Sl.No	Field Name	Data Type	Size	Constraints
1	Deadline_id	VARCHAR	55	Primary Key
2	batch_id	VARCHAR		Foreign Key
3	title	VARCHAR	150	Not null
4	description	TEXT		
5	Due_date	DATE		NOT NULL

11. Table: Rubrics

Description: Stores rubric definitions created by coordinators for evaluation.

Sl.No	Field Name	Data Type	Size	Constraints
1	Rubric_id	VARCHAR	55	Primary Key
2	deadline_id	VARCHAR		Foreign Key
3	Max_score	INTEGER		NOT NULL
4	criteria	JSONB		NOT NULL

12. Table: Evaluation Scores

Description: Stores rubric-based evaluation scores for each group.

Sl.No	Field Name	Data Type	Size	Constraints
1	Eval_id	VARCHAR	55	Primary Key
2	Rubric_id	VARCHAR		Foreign Key
3	Group_id	VARCHAR		Foreign Key
4	Evaluated_by	VARCHAR		Foreign Key
5	Score	INTEGER		
6	Criteria	JSONB		

4.4 PROCESS DESIGN

Process design defines the workflow of the Academic Project Monitoring System to ensure efficient project supervision and seamless interaction between students, guides, and coordinators. The system operates through a structured flow of data collection, processing,

approval, and reporting.

When project topics are submitted by students, they are routed to the assigned guide for review and approval. Once approved, the coordinator is notified and the project progresses through defined phases such as SRS, Mid-term Review, and Final Submission. Milestones and deadlines are configured by the coordinator and are visible to all stakeholders.

Guides can review submitted documents, provide inline feedback, and update approval status for each version. Students can access their project dashboard, manage tasks on the Agile board, upload new document versions, and schedule meetings. During the review process, guides record interventions and track student progress over time.

The system continuously updates data in real time, maintaining logs for transparency and accountability. Notifications are sent to students and Guide regarding performance updates, session schedules, and important alerts. This structured workflow ensures proactive academic monitoring, timely intervention, and improved coordination among all users, making the overall academic management process more efficient and intelligent.

-organized.

MODULE DESCRIPTION

Student

- Register and log in securely to the system
- Submit and track project topics (Pending/Approved/Rejected status)
- Manage project tasks on a Kanban-style board (To Do / In Progress / Done)
- Upload versioned documents (proposal, review docs, final report)
- Receive real-time alerts and notifications
- View guide feedback and rubric-based evaluation scores
- Update basic profile information
- Access a user-friendly dashboard

Guide

- Log in securely to the guide portal

- View assigned students and their academic details
- Monitor student performance and AI risk status
- Receive alerts for at-risk students
- Approve, reject, or request revision on project topics and documents
- Define milestones and configure review phases for assigned groups
- Evaluate student progress and outcomes
- Communicate with students
- Export student group progress reports in PDF or Excel format

Coordinator

- Log in securely with role-based access
- Manage students, Guide, courses, and batches
- Configure batches, set global deadlines, and manage rubric criteria
- Monitor system performance and academic analytics
- Monitor project delays, risk alerts, and guide workload balance
- Manage notifications and system updates
- Handle user issues and system configurations
- Maintain audit logs and ensure data integrity

AI Engine

- Allocate guides to student groups and balance faculty workload
- Monitor department-level project progress and compliance
- Identify at-risk students automatically
- Generate alerts for Guide
- Approve final project submissions and archive completed projects
- Support data-driven academic decision-making
- Continuously update predictions based on new data

SYSTEM TESTING & IMPLEMENTATION

5. SYSTEM TESTING & IMPLEMENTATION

4.1 SYSTEM TESTING

System testing ensures that the Academic Project Monitoring System functions correctly, meets all specified requirements, and provides a smooth and reliable experience for users. Various testing methods are applied to validate different modules such as authentication, project management, document versioning, Agile task boards, evaluation workflows, real-time communication, and notification systems. These tests help ensure system reliability, security, and performance under real-world academic conditions. After successful testing, the system is implemented and deployed for practical use within the institution.

Testing technique

- Unit testing
- Integration Testing
- Validation Testing
- System Testing
- Output Testing
- User Acceptance Testing

5.1.1 Unit Testing

Unit testing focuses on testing individual modules or components of the Academic Project Monitoring System in isolation to ensure that each unit functions correctly as expected. Modules such as user authentication, topic submission, document upload and versioning, task management, milestone tracking, rubric-based evaluation, meeting scheduling, and notification systems are tested separately. Developers design test cases to validate inputs, verify error handling, and ensure that each functionality performs accurately before integrating it into the complete system.

Test Case	Input	Expected Output	Actual Output	Result
Student login (valid)	Email + Password	JWT token generated	Token generated	Pass
Student login (invalid)	Wrong password	Error message	Error displayed	Pass
Student uploads synopsis document	Valid PDF file selected	Document saved, version 1 created, status set to Pending	Document uploaded, version recorded	Pass
Submit project topic for approval	Title, domain, description entered	Topic saved with status Pending, guide notified	Topic submitted, notification sent to guide	Pass
Test Case	Input	Expected Output	Actual Output	Result

5.1.2 Integration Testing

Integration testing ensures that different modules of the Academic Project Monitoring System work together seamlessly as a unified platform. Since the system includes multiple components such as the React frontend, Node.js/Express API layer, FastAPI services, PostgreSQL database, and authentication system, it is important to verify proper data flow and communication between them. This testing focuses on key workflows such as topic approval, document review, milestone progression, meeting scheduling, evaluation scoring, and notification handling, ensuring that all modules interact correctly.

It also verifies the interaction between different user roles — students, guides, and coordinators

— ensuring that each role receives accurate data and appropriate access permissions. Integration testing ensures that features such as authentication, document versioning, evaluation workflows, real-time chat, and deadline notifications function properly across the system without conflicts.

Test Case	**Input**	**Expected Output**	**Actual Output**	**Result**
Fetch student profile	Student login	Profile displayed	Data retrieved	Pass
Guide views assigned groups	Guide logs in and opens dashboard	All assigned groups and their project status displayed	Group list rendered correctly	Pass
Guide approves uploaded document	Guide clicks Approve on a Pending document	Document status updated to Approved, student notified	Status changed, in-app notification delivered to student	Pass
Student requests meeting with guide	Date, time, agenda submitted by student	Meeting record created, guide receives meeting request notification	Meeting saved, guide notified successfully	Pass
Test Case	Input	Expected Output	Actual Output	Result

5.1.3 Validation Testing

Validation testing ensures that the Academic Project Monitoring System meets all functional and system requirements as defined. It verifies that coordinators can manage batches, configure rubrics, allocate guides, and monitor departmental progress effectively. It also ensures that guides can review documents, provide inline feedback, approve topics, define milestones, and rate student groups efficiently, while students can submit topics, upload documents, manage tasks, and schedule meetings without issues.

Test cases are designed based on system requirements to confirm that all features, including document versioning, rubric-based evaluation, Agile task boards, and notification systems, perform accurately and as expected.

Test Case	Input	Expected Output	Actual Output	Result
Topic submission without title	Empty title field submitted	Error message	Error displayed	Pass
Guide approves valid project topic	Guide clicks Approve on a Pending topic	Topic status updated to Approved, student notified	Registered successfully	Pass
Upload non-PDF file as project document	Image file (.jpg) uploaded in document field	File type rejected, error message shown	Error displayed	Pass
Coordinator sets deadline for a batch	Due date, phase, title entered and saved	Deadline created and visible to all groups in that batch	Registered successfully	Pass
Test Case	Input	Expected Output	Actual Output	Result

5.1.4 System Testing

System testing evaluates the overall performance, security, and functionality of the Academic Project Monitoring System under real-world conditions. It includes load testing, stress testing, and security testing to ensure that the system can handle multiple students, guides, and coordinators accessing the platform simultaneously. The testing verifies smooth execution of key processes such as topic approval workflows, document upload and versioning, milestone tracking, evaluation scoring, and notification delivery. It also ensures that role-based access control is properly enforced and unauthorized users cannot access restricted modules.

The system is tested under high user load to ensure stability, fast data retrieval, and reliable performance. Real-world academic project scenarios are simulated to confirm that all modules, including the Kanban task board, document review workflows, and analytics dashboards, work seamlessly together without delays or failures, ensuring a stable and efficient academic project monitoring system.

.

Test Case	Input	Expected Output	Actual Output	Result
Concurrent users accessing system	100 simultaneous student/guide logins	System stable, response time under 2 seconds	No crash or timeout observed	Pass
Student attempts to access coordinator panel	Student navigates to /coordinator/dashboard	Access denied, redirected to student dashboard	Route blocked, RBAC enforced correctly	Pass
Multiple groups submit documents simultaneously	20 groups upload documents at the same time	All documents saved, versions recorded without conflict	All uploads processed successfully	Pass
End-to-end topic approval workflow	Student submits topic → Guide approves → Coordinator notified	All three stages completed, statuses updated correctly	Full workflow executed without errors	Pass

5.1.5 Output Testing

Output testing verifies that all system outputs in the Academic Project Monitoring System are generated correctly and presented in a clear, accurate, and user-friendly manner. The system must provide correct outputs such as project progress reports, document review status, rubric-based evaluation scores, meeting summaries, and deadline notifications. It ensures that coordinators can view department analytics and audit logs, guides can access group performance reports and document feedback histories, and students can view their project progress and evaluation scores without errors.

Testers also verify that dashboards, evaluation reports, exported documents, and notifications are properly formatted, consistent, and easy to understand. This ensures that all users receive reliable and meaningful information, enabling effective project supervision and academic decision-making.

Test Case	Input	Expected Output	Actual Output	Result
Deadline notification to students	Coordinator sets a deadline for a batch	All students in the batch receive in-app deadline notification	Notification displayed on student dashboard	Pass
Guide exports group progress report	Guide clicks Export for a group	PDF/Excel report generated with task and milestone data	Report downloaded successfully with correct data	Pass
Student receives meeting confirmation	Guide approves student's meeting request	Student receives confirmation notification with date and time	Displayed	Pass
Guide rejects uploaded document with feedback	Guide clicks Reject and enters inline comment	Document status set to Rejected, feedback saved and visible to student	Rejection and feedback displayed on student's document page	Pass
Test Case	Input	Expected Output	Actual Output	Result

5.1.6 User Acceptance Testing (UAT)

User Acceptance Testing (UAT) involves real users — students, guides, and coordinators — interacting with the Academic Project Monitoring System to ensure it meets their expectations and requirements. Coordinators test functionalities such as batch configuration, guide allocation, deadline management, and department-level analytics. Guides test features like reviewing project topics, annotating documents, tracking group milestones, and scoring evaluations, while students verify functionalities such as submitting topics, uploading documents, managing Agile task boards, scheduling meetings, and receiving notifications.

Feedback is collected from all users to identify usability issues, improve user experience, and address any missing features before final deployment. This phase ensures that the system is user-friendly, efficient, and suitable for real-world academic institutions, providing smooth project coordination and reliable performance for all stakeholder roles.

Test Case	Input	Expected Output	Actual Output	Result
Student dashboard	Student logs in and opens project overview	Project title, progress, tasks, deadlines, and feedback visible	Displayed	Pass
Guide reviews and annotates SRS document	Guide opens document and adds inline comment	Comment saved, document status updated, student notified	Feedback visible to student, status reflects review	Pass
Student creates task on Kanban board	Student adds task with title, priority, and deadline	Task appears on Kanban board under To Do column	Task created and visible to guide on group Kanban view	Pass
Coordinator views departm	Coordinator opens department overview	Batch-wise project status,	Displayed	Pass

ent analytics dashboard		guide workload, and risk alerts displayed		

5.2 SYSTEM IMPLEMENTATION

System implementation is the final and most crucial phase of the Academic Project Monitoring System project, where the developed system is deployed for real-world project supervision and management. This phase ensures that all modules function correctly in a live environment and that users can effectively interact with the system. It involves system testing, deployment, user onboarding, and ensuring smooth operational functionality across all roles such as students, guides, and coordinators. A well-planned implementation ensures a smooth transition from traditional manual project management to a fully digital and transparent platform.

The implementation process begins with preparing the system environment, including server setup, database configuration, and application deployment. The frontend is developed using React, while the backend is implemented using Node.js with Express and FastAPI, and PostgreSQL is used for secure and structured data storage. Proper configuration ensures seamless communication between all components, enabling efficient handling of operations such as topic approval workflows, document version management, milestone tracking, rubric-based evaluation, and notification delivery.

Before deployment, the system undergoes rigorous testing to ensure reliability and accuracy. Various testing techniques such as unit testing, integration testing, system testing, and user acceptance testing are performed. Real-time testing is conducted to validate core functionalities such as topic approval workflows, document versioning, task management, milestone tracking, and notification delivery. Any identified bugs or performance issues are resolved to ensure smooth system operation under real-world academic conditions.

The system is deployed in a phased manner to reduce risks and ensure stability. Core modules such as authentication, project management, document handling, task boards, communication, and evaluation workflows are implemented step by step. This approach helps verify each module independently and ensures that the complete system functions without conflicts during full deployment.

User onboarding and training are essential aspects of the implementation phase. Students, guides, and coordinators are provided with proper guidance on how to use the system. Training sessions, documentation, and user-friendly interfaces help users quickly adapt to the platform. This

minimizes resistance to change and improves overall system effectiveness.

Data migration is another important step, where existing academic data such as student records, group assignments, and historical project information (if available) are transferred from manual or legacy systems into the system database. Proper validation is performed to ensure data accuracy and consistency, allowing the system to operate reliably from the beginning.

After deployment, continuous monitoring and evaluation are carried out to ensure system performance, security, and reliability. System logs, user activity, and performance metrics are monitored to detect and resolve issues promptly. User feedback is collected to identify improvements and enhance system usability.

The implementation phase also includes planning for future scalability and enhancements. Regular updates, performance optimization, and feature improvements are carried out to meet evolving academic needs. By following a structured implementation approach, the Academic Project Monitoring System provides a reliable, efficient, and scalable solution for academic project supervision, ensuring improved project quality, better guide-student coordination, and overall system effectiveness.

.

SECURITY TECHNOLOGIES & POLICIES

6. SECURITY TECHNOLOGIES & POLICIES

Security plays a critical role in ensuring the confidentiality, integrity, and availability of data in the Academic Project Monitoring System. Since the platform handles sensitive academic information such as project documents, evaluation scores, student records, communication logs, and user credentials, strong security measures are essential to prevent unauthorized access, data breaches, and system misuse. A multi-layered security approach is implemented to safeguard data at every level and provide a secure and reliable user experience.

To protect data transmission, the system uses Secure Socket Layer (SSL) and Transport Layer Security (TLS) protocols, ensuring that all communication between the client and server is encrypted. At the database level, security mechanisms such as access control, constraints, and validation are implemented to prevent unauthorized access and data manipulation. These measures ensure that sensitive academic and user data is securely stored and managed.

Authentication and access control are strictly enforced using JSON Web Tokens (JWT) for secure session management. The system follows Role-Based Access Control (RBAC), ensuring that users such as students, guides, and coordinators can only access functionalities relevant to their roles. Passwords are securely stored using hashing algorithms like bcrypt, preventing exposure of plain-text credentials. Additional mechanisms such as token expiration and secure login practices further strengthen system security.

To prevent cyber threats such as unauthorized access, SQL injection attacks, cross-site scripting (XSS), and cross-site request forgery (CSRF), the system incorporates strict input validation, parameterized database queries, API rate limiting, and secure coding practices across both the Node.js/Express and FastAPI backends. Server-side middleware and monitoring tools help detect unusual access patterns, while session management policies such as automatic JWT expiry and logout after inactivity reduce the risk of unauthorized access. These measures ensure the system remains resilient against common web vulnerabilities.

Data protection is further enhanced through regular backup and recovery strategies. The system maintains automated daily backups of the PostgreSQL database — including all project records, documents, evaluation scores, and communication logs — to prevent data loss due to system failures or cyberattacks. A disaster recovery plan ensures quick restoration of services with

minimal downtime, maintaining continuity in academic project supervision activities.

Monitoring and logging mechanisms track system activities such as login attempts, topic submissions, document uploads, evaluation actions, meeting requests, and chat messages. These activity logs provide full audit trails for accountability across all three user roles. Real-time alerts notify coordinators of potential security threats or unusual access patterns, enabling quick response and maintaining system integrity.

User awareness is also emphasized to reduce risks associated with human errors. The system enforces strong password policies and encourages users to follow secure practices such as protecting credentials and avoiding unauthorized access. Clear data handling policies ensure that all academic and personal information is managed with strict confidentiality.

By implementing these comprehensive security technologies and policies, the Academic Project Monitoring System ensures a secure, reliable, and trustworthy academic project supervision platform, protecting sensitive data while enabling efficient and safe system operations across all stakeholder roles.

.

MAINTENANCE

7. MAINTENANCE

Software maintenance is essential to ensure the long-term reliability, security, and efficiency of the Academic Project Monitoring System. After deployment, continuous monitoring, updates, and enhancements are required to meet evolving institutional requirements and technological advancements. A structured maintenance approach is followed to keep the system stable, scalable, and aligned with the goal of efficient academic project supervision.

Corrective maintenance focuses on identifying and resolving system errors, bugs, and technical issues reported by students, guides, or coordinators. Issues related to document upload, task management, notification delivery, evaluation workflows, or user authentication are promptly addressed. Regular debugging and troubleshooting help maintain system stability and prevent disruptions in academic project activities.

Adaptive maintenance ensures that the system remains compatible with changing technologies and institutional requirements. As new needs arise, such as enhanced analytics dashboards, improved plagiarism detection, or integration with university ERP systems, updates are implemented to improve system functionality. These enhancements help keep the system modern, user-friendly, and relevant.

Preventive maintenance is carried out to reduce the risk of future system failures. Continuous monitoring of server performance, database operations, and application health helps identify potential issues early. Regular updates, performance tuning, and system audits are performed to ensure smooth and uninterrupted operation.

Performance optimization is important to maintain a fast and responsive system, especially when handling multiple users and real-time operations. Techniques such as optimized database queries, caching mechanisms, and efficient resource management are used to ensure quick data retrieval and smooth user interaction without delays.

Security maintenance plays a vital role in protecting sensitive academic and personal data. Regular security updates, vulnerability assessments, and improvements in authentication mechanisms are implemented to prevent unauthorized access and cyber threats. Encryption, secure session management, and monitoring tools are continuously updated to maintain a secure system environment.

Database maintenance ensures efficient storage, retrieval, and integrity of data such as project records, document versions, evaluation scores, meeting logs, and communication histories. Regular backups, indexing, and validation processes are performed to prevent data loss, corruption, and redundancy. Backup mechanisms ensure quick recovery in case of system failures.

User support is also an important aspect of maintenance. Students, guides, and coordinators are provided with support through documentation, help guides, and assistance services. Feedback collected from users is used to improve system usability and introduce new features.

By implementing continuous and structured maintenance strategies, the Academic Project Monitoring System remains a reliable, secure, and high-performing academic project supervision platform, ensuring improved user experience, system stability, and long-term sustainability.

CONCLUSION

8. CONCLUSION

The Academic Project Monitoring System is an innovative and technology-driven platform designed to streamline and digitize the complete lifecycle of academic project supervision within educational institutions. By providing a centralized platform where coordinators can manage batches and guide allocations, guides can supervise project groups and provide structured feedback, and students can manage tasks, upload deliverables, and track progress, the system significantly enhances efficiency, transparency, and overall academic project quality. The use of modern technologies such as React, Node.js, FastAPI, and PostgreSQL ensures smooth performance, scalability, and secure data handling.

The system effectively connects students, guides, and coordinators, enabling seamless coordination and structured workflows throughout the academic project lifecycle. Features such as milestone management, versioned document upload, inline feedback, Agile task boards, rubric-based evaluation, real-time communication, and department-level analytics reduce manual effort and provide complete visibility into project progress. This proactive approach improves guide effectiveness, student accountability, and overall academic outcomes.

With strong security mechanisms, continuous maintenance strategies, and a scalable architecture, the Academic Project Monitoring System is adaptable to future technological advancements and institutional requirements. The platform provides a reliable, efficient, and user-friendly solution for academic project supervision, ensuring better decision-making, improved project quality, and an enhanced academic experience for all stakeholders.

8.1 SCOPE FOR FUTURE ENHANCEMENTS

The Academic Project Monitoring System has strong potential to evolve into a more comprehensive and intelligent academic project supervision platform. The following enhancements can further improve its capabilities and institutional impact:

1. Advanced AI & Predictive Analytics

Future development can incorporate AI-driven features to provide deeper project insights and more accurate analytics.

- Predict project completion likelihood based on milestone progress and task activity
- Recommend guide allocation strategies based on expertise and workload patterns

2. Intelligent Supervision Support

The system can be enhanced to support smarter project supervision processes.

- AI-driven risk alerts for project delays and inactivity
- Automated generation of project health reports for coordinators

3. Mobile Application & Accessibility

The system can be extended to mobile platforms for improved accessibility.

- Dedicated mobile applications for students and guides
- Real-time notifications and easy access to dashboards
- Offline support with data synchronization

4. Advanced Analytics & Visualization

The system can provide deeper insights through enhanced reporting.

- Interactive dashboards for project health, milestone completion, and evaluation trends
- Visual representation of group progress across phases and sprints
- Institution-level project quality and compliance analytics

5. Integration with Academic Systems

The Academic Project Monitoring System can be integrated with existing institutional platforms.

- Integration with university ERP systems and Learning Management Systems (LMS)
- Automated synchronization of student enrollment and academic records
- Seamless data exchange across systems

6. Plagiarism Detection Enhancement

The document review process can be strengthened with advanced plagiarism analysis.

- Integration with academic plagiarism detection APIs for automated document scanning
- Similarity score reports generated automatically upon document upload

7. AI Chat Assistant

Enhance the system with an intelligent assistant.

- Provide instant guidance to students on project requirements and deadlines
- Answer queries related to project phases, document submission guidelines, and evaluation criteria

BIBLIOGRAPHY

9. BIBLIOGRAPHY

The following references were used for the development and understanding of technologies used in the Academic Project Monitoring System:

- React Documentation. (2024). React Official Documentation. https://react.dev/
- Node.js Documentation. (2024). Node.js Official Documentation. https://nodejs.org/en/docs
- Express Documentation. (2024). Express.js Official Documentation. https://expressjs.com/
- FastAPI Documentation. (2024). FastAPI Official Documentation. https://fastapi.tiangolo.com/
- Tailwind CSS Documentation. (2024). Utility-First CSS Framework. https://tailwindcss.com/
- PostgreSQL Documentation. (2024). PostgreSQL 16 Documentation. https://www.postgresql.org/docs/
- JWT.io. (2024). JSON Web Tokens - Introduction. https://jwt.io/introduction
- Geeksforgeeks. (2024). Full Stack Web Development with Node.js and React. https://www.geeksforgeeks.org/
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
- Pressman, R. S. (2014). Software Engineering: A Practitioner’s Approach (8th ed.). McGraw-Hill.

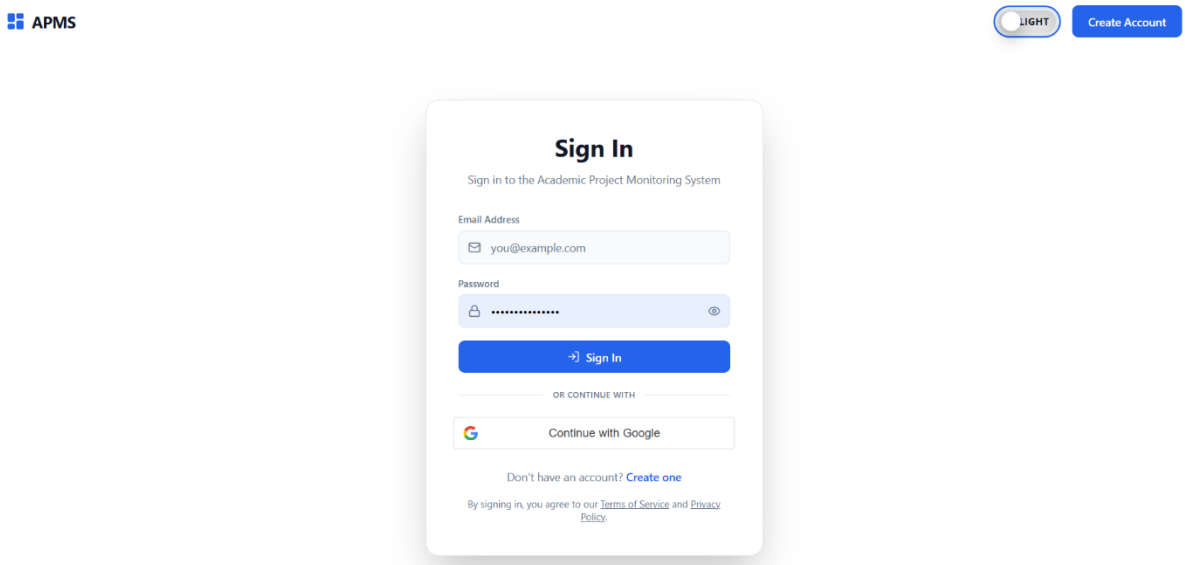
APPENDIX

10. APPENDIX

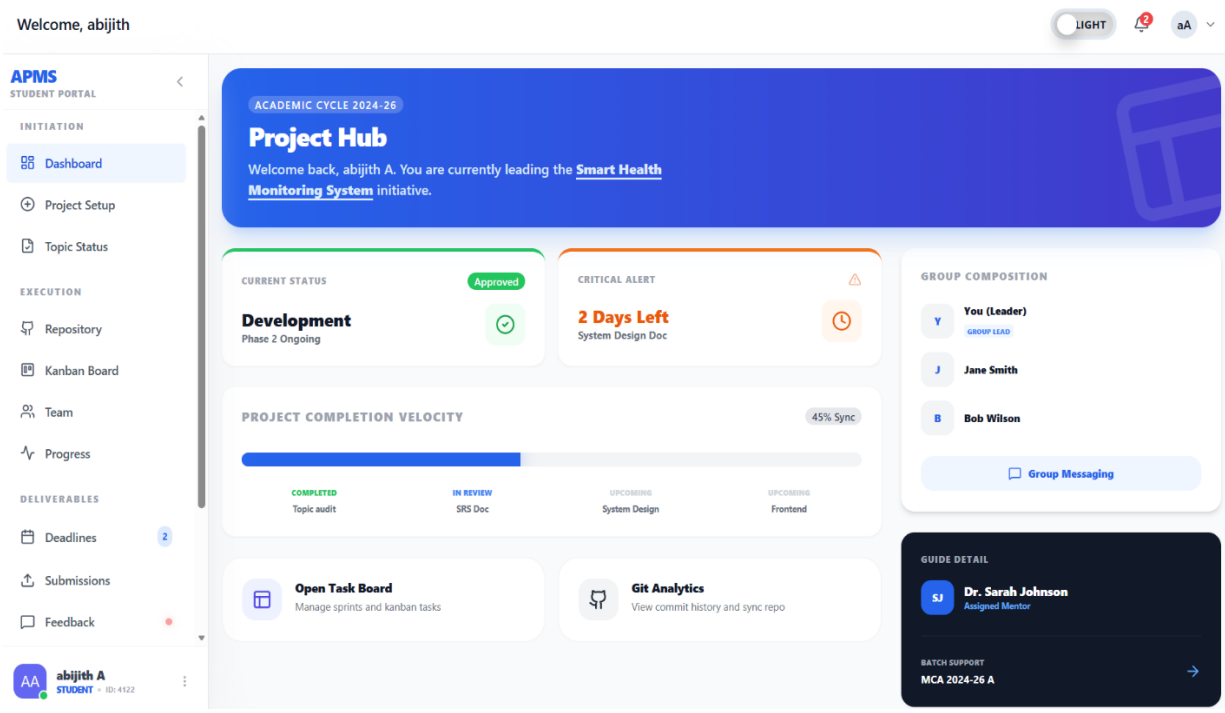
10.1 SCREEN SHOTS

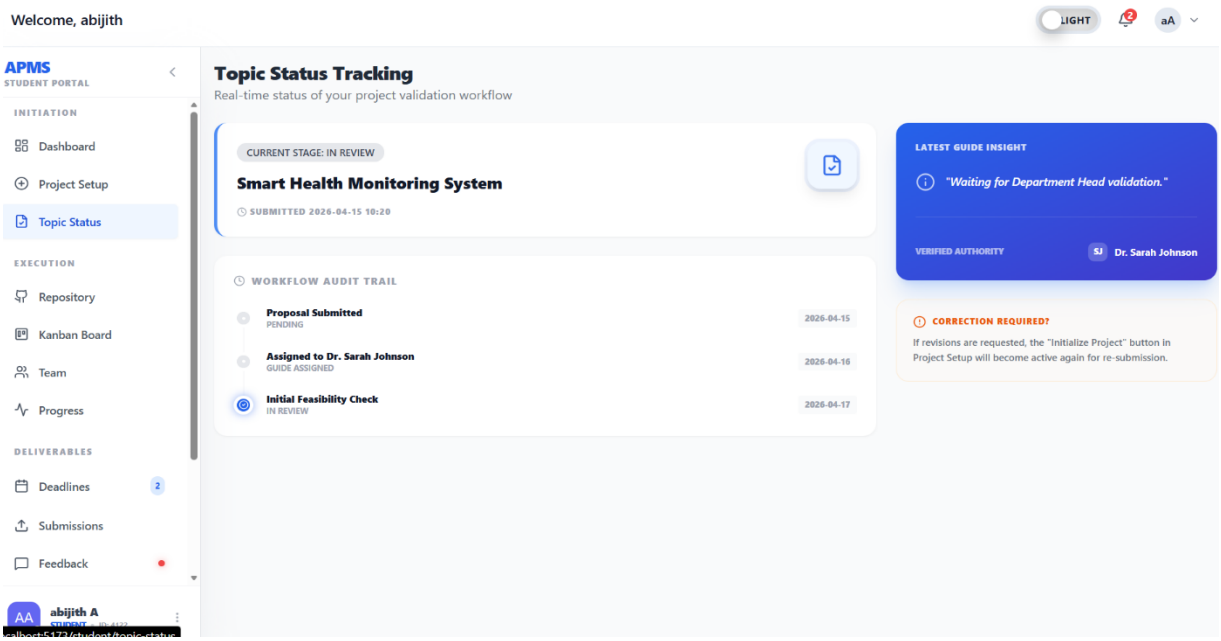
Landing Page

Login

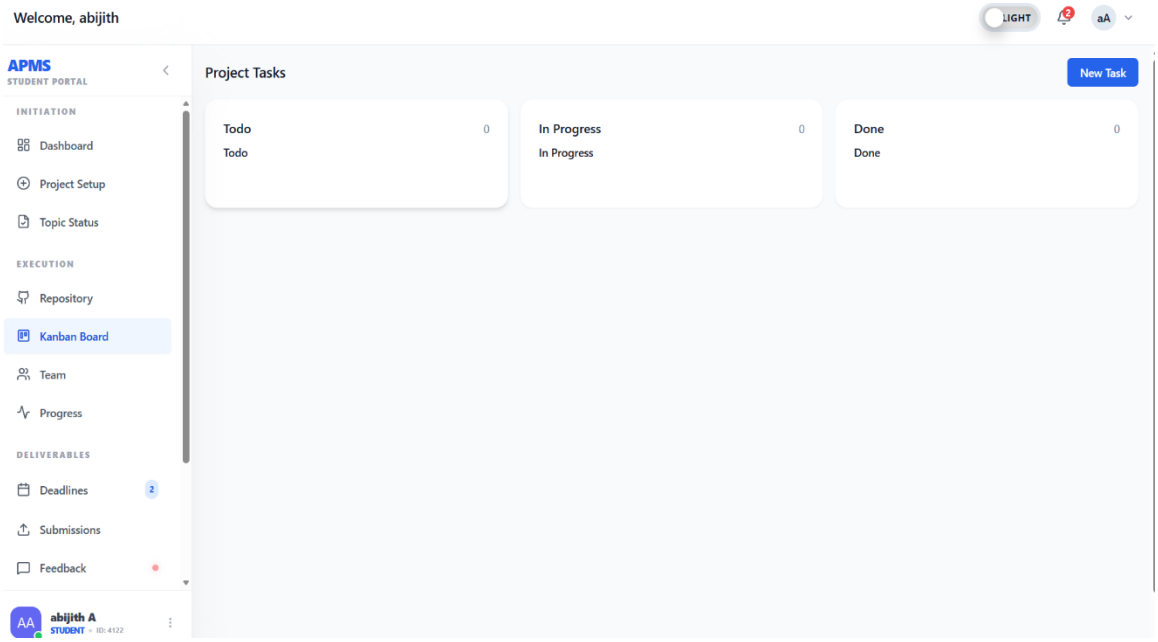


Student Dashboard

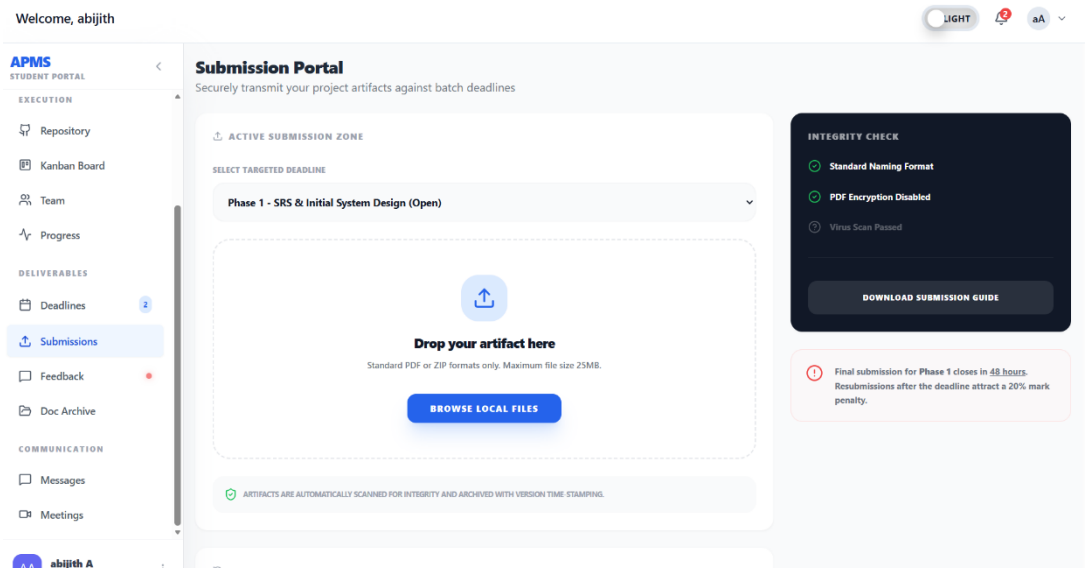




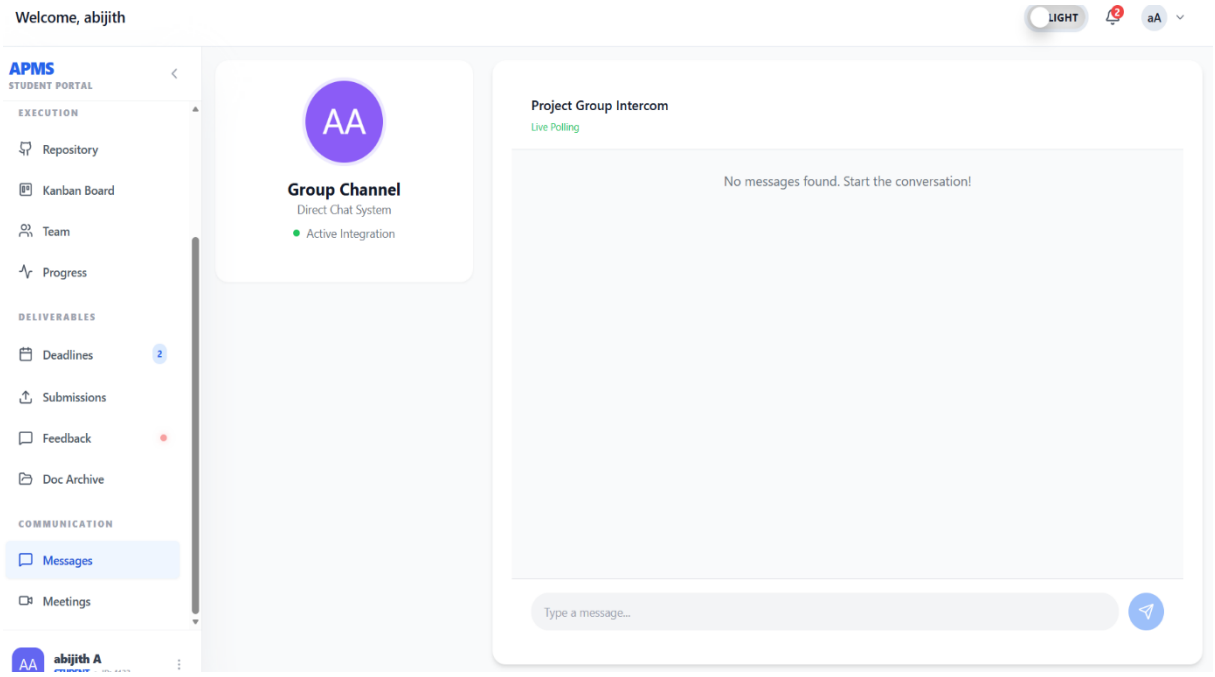
Student Task Board



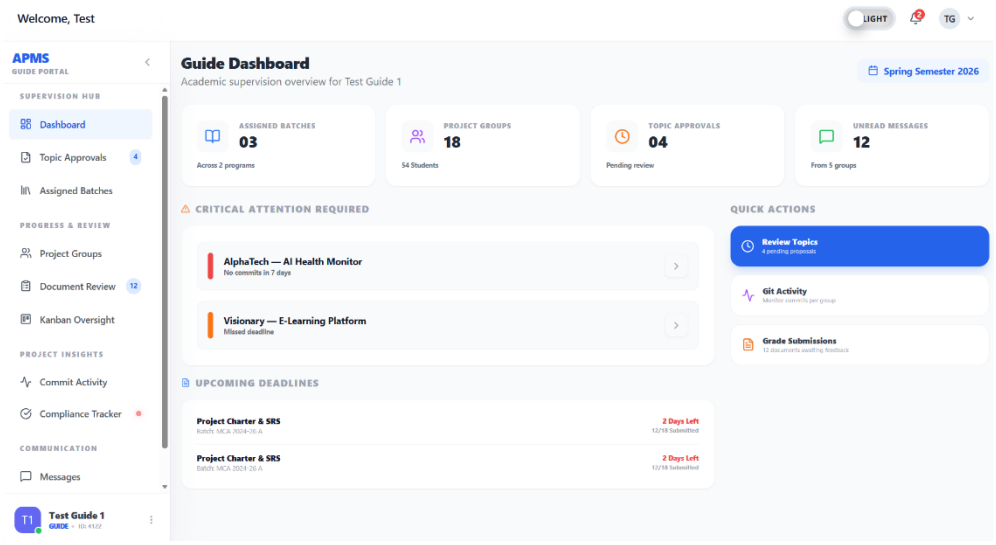
Submission Portal



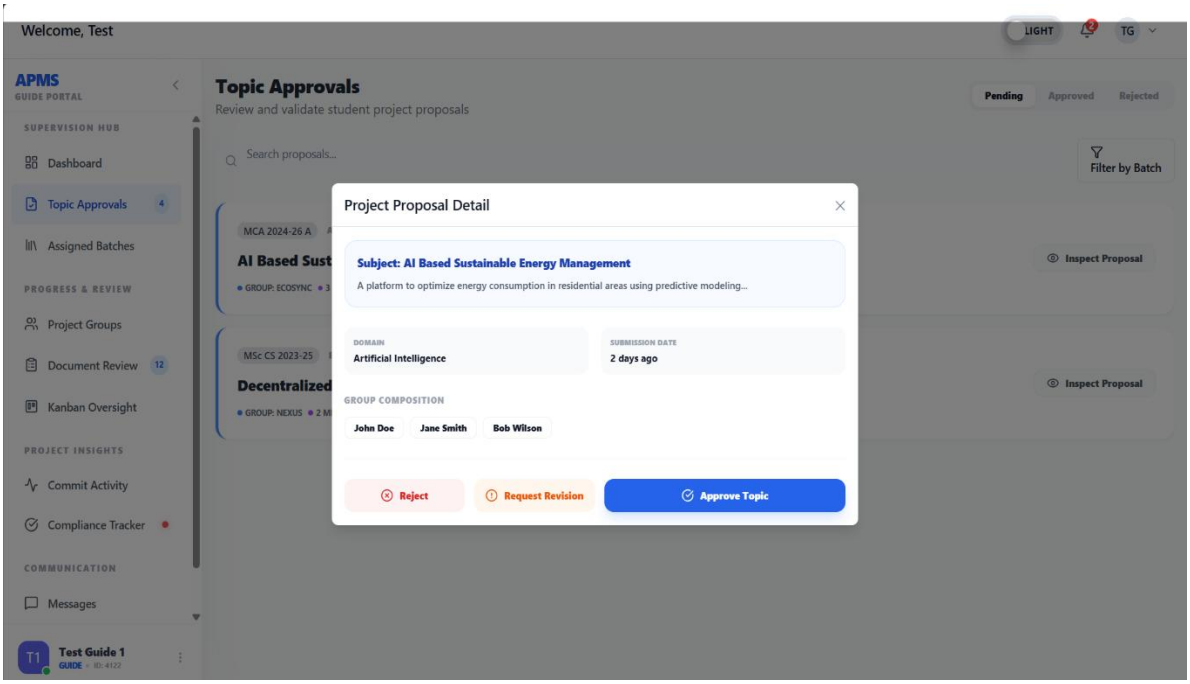
Chat-session



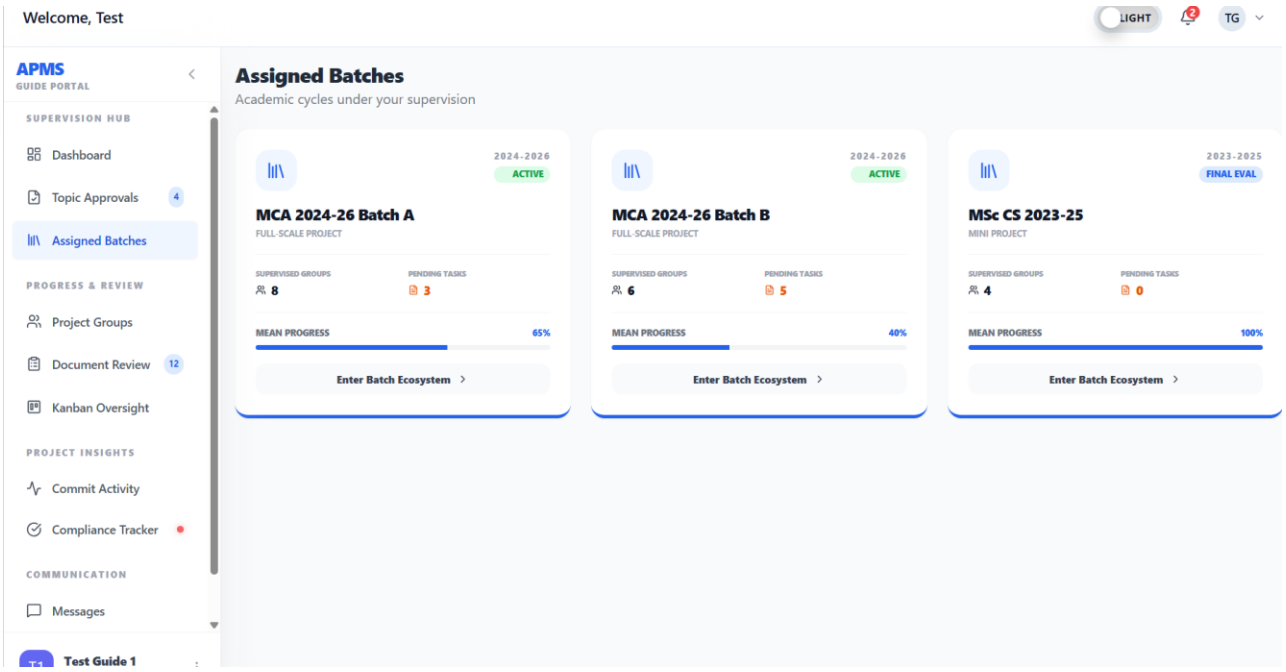
Guide Dashboard



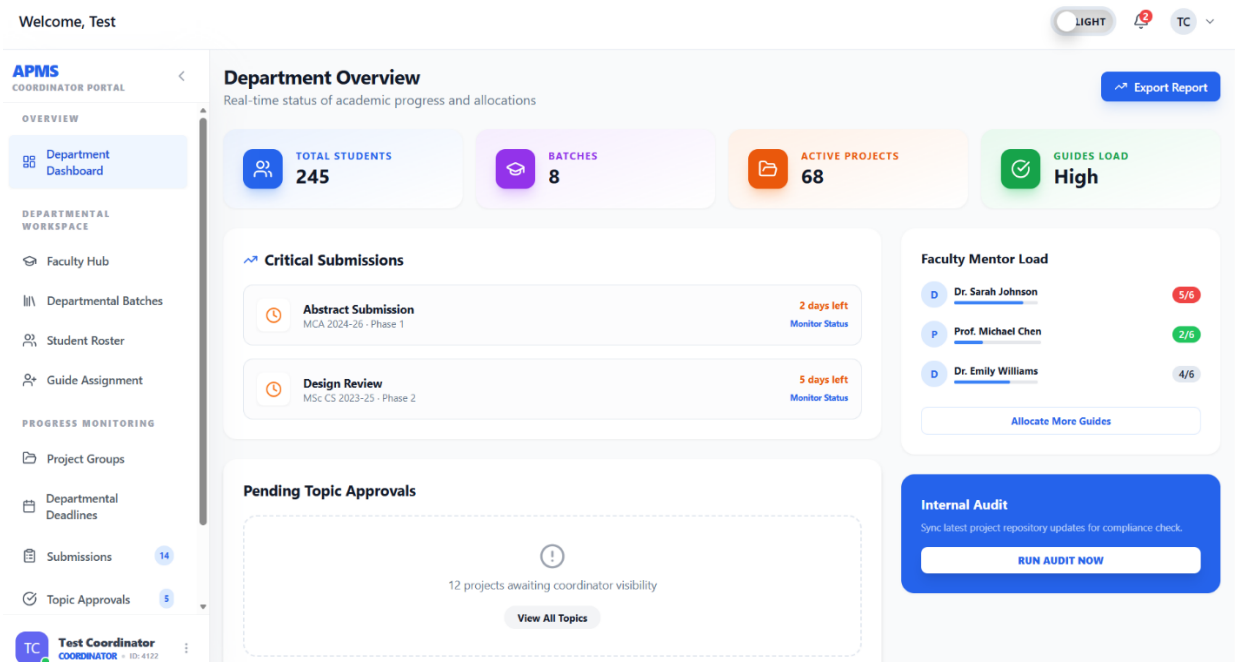
Topic Approvals



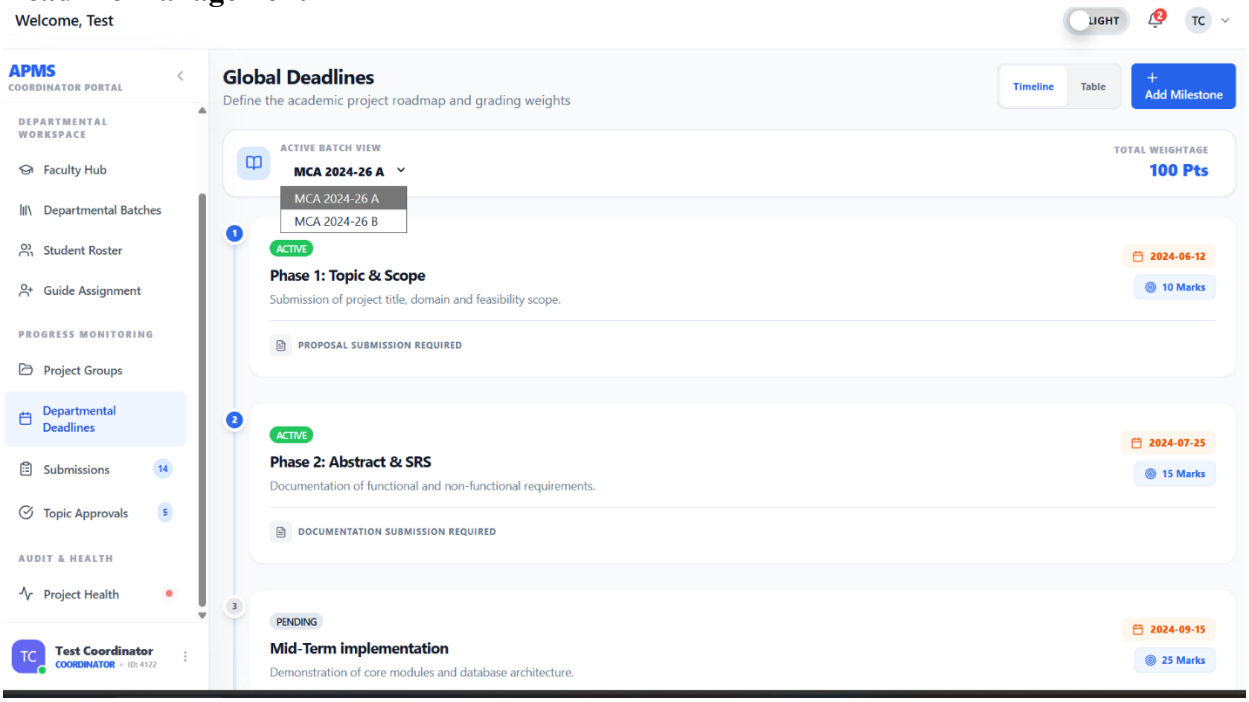
Assigned-Batches



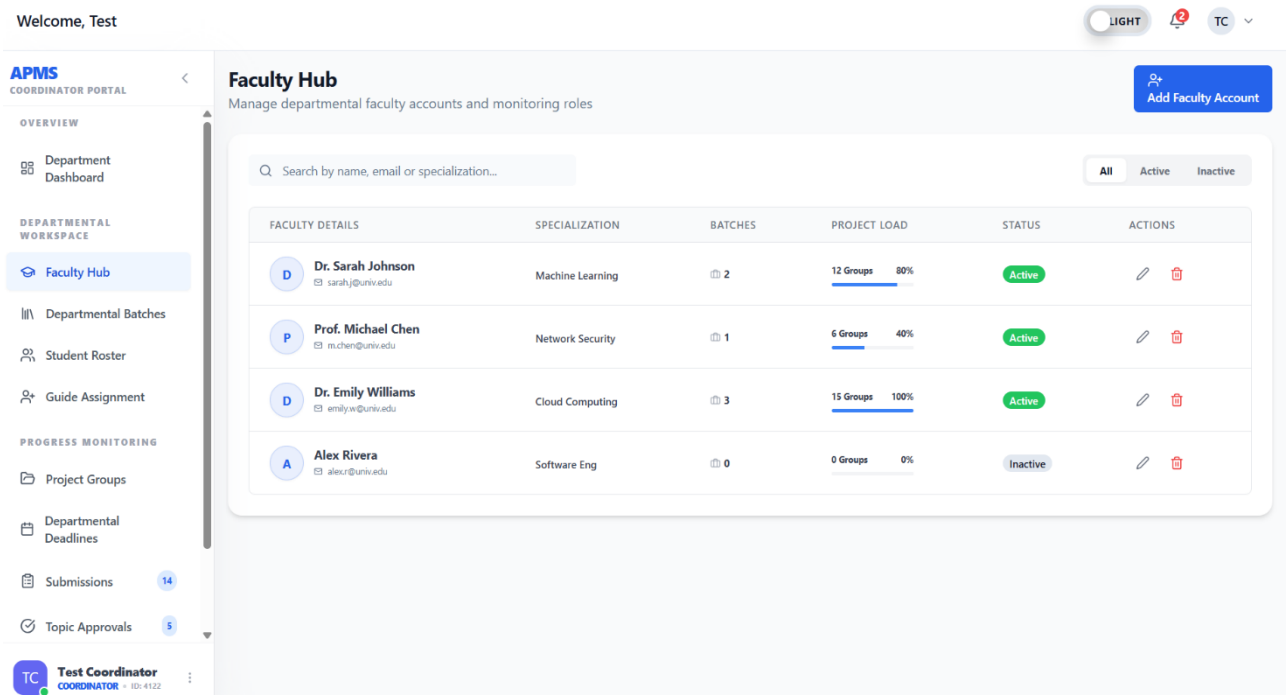
Coordinator Dashboard



Deadline management



Faculty Management



10.2 CODE

```
import React from 'react';
import { useAuth } from '../../hooks/useAuth';
import Card from '../../components/common/UI/Card';
import Badge from '../../components/common/UI/Badge';
import {
  Github, Layout, CheckCircle2, AlertTriangle,
  Clock, MessageSquare, ArrowRight
} from 'lucide-react';
import { useNavigate } from 'react-router-dom';
```

```
const StudentDashboard: React.FC = () => {
  const { user } = useAuth();
  const navigate = useNavigate();

  // High fidelity mock for a student with a project
  const project = {
    title: 'Smart Health Monitoring System',
    status: 'Approved',
    progress: 45,
    mode: 'Group',
    batch: 'MCA 2024-26 A',
    guide: 'Dr. Sarah Johnson',
    members: ['You (Leader)', 'Jane Smith', 'Bob Wilson'],
    repo: 'alphatech/shm-project',
    nextDeadline: {
      name: 'System Design Doc',
      date: 'April 20, 2026',
      daysLeft: 2
    }
  };

  return (
    <div className="space-y-6">
      {/* Header with Background Gradient */}
      <div className="relative p-8 rounded-3xl overflow-hidden bg-gradient-to-r from-blue-600
to-indigo-700 text-white shadow-xl shadow-blue-500/20">
        <div className="relative z-10">
          <div className="flex items-center gap-2 mb-2">
            <Badge className="bg-white/20 text-white border-none text-[9px] font-black
uppercase tracking-widest">
              Academic Cycle 2024-26
            </Badge>
          </div>
```

```
            <h1 className="text-3xl font-black tracking-tight">Project Hub</h1>
            <p className="text-blue-100 mt-2 font-medium max-w-lg">
              Welcome back, {user?.name}. You are currently leading the <span className="font-
black underline decoration-2 underline-offset-4">{project.title}</span> initiative.
            </p>
          </div>
          {/* Decorative background icon */}
          <Layout size={180} className="absolute top-1/2 right-0 -translate-y-1/2 opacity-10 -
rotate-12 translate-x-12" />
        </div>

        <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
          {/* Left: Project Stats & Progress */}
          <div className="lg:col-span-2 space-y-6">
            <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
              <Card className="border-t-4 border-t-green-500">
                <div className="flex items-center justify-between mb-4">
                  <p className="text-[10px] font-black text-gray-400 uppercase tracking-
widest">Current Status</p>
                  <Badge variant="success" className="text-[10px] font-
black">{project.status}</Badge>
                </div>
                <div className="flex items-end justify-between">
                  <div>
                    <h3 className="text-xl font-black">Development</h3>
                    <p className="text-xs text-gray-500 font-bold">Phase 2 Ongoing</p>
                  </div>
                  <div className="p-3 bg-green-50 dark:bg-green-900/20 text-green-600 rounded-
xl">
                    <CheckCircle2 size={24} />
                  </div>
                </div>
              </Card>
```

```
            <Card className="border-t-4 border-t-orange-500">
              <div className="flex items-center justify-between mb-4">
                <p className="text-[10px] font-black text-gray-400 uppercase tracking-
widest">Critical Alert</p>
                <span className="text-orange-600 animate-pulse"><AlertTriangle size={16}
/></span>
              </div>
              <div className="flex items-end justify-between">
                <div>
                  <h3 className="text-xl font-black text-orange-
600">{project.nextDeadline.daysLeft} Days Left</h3>
                  <p className="text-xs text-gray-500 font-
bold">{project.nextDeadline.name}</p>
                </div>
                <div className="p-3 bg-orange-50 dark:bg-orange-900/20 text-orange-600
rounded-xl">
                  <Clock size={24} />
                </div>
              </div>
            </Card>
          </div>

          <Card className="relative">
            <div className="flex items-center justify-between mb-8">
              <h2 className="text-sm font-black uppercase tracking-widest text-gray-
400">Project Completion Velocity</h2>
              <Badge variant="secondary" className="font-black">{project.progress}%
Sync</Badge>
            </div>

            <div className="space-y-6">
              <div className="h-4 w-full bg-gray-100 dark:bg-gray-800 rounded-full overflow-
hidden">
                <div className="h-full bg-blue-600 shadow-[0_0_20px_rgba(37,99,235,0.4)]"
```

```
style={{ width: `${project.progress}%` }}></div>
            </div>

            <div className="grid grid-cols-2 md:grid-cols-4 gap-4">
              {[
                { label: 'Topic audit', status: 'Completed', color: 'text-green-500' },
                { label: 'SRS Doc', status: 'In Review', color: 'text-blue-500' },
                { label: 'System Design', status: 'Upcoming', color: 'text-gray-300' },
                { label: 'Frontend', status: 'Upcoming', color: 'text-gray-300' },
              ].map(step => (
                <div key={step.label} className="text-center">
                  <div className={`text-[9px] font-black uppercase mb-1
${step.color}`}>{step.status}</div>
                  <p className="text-[10px] font-bold text-gray-500">{step.label}</p>
                </div>
              ))}
            </div>
          </div>
        </Card>

        <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
          <button onClick={() => navigate('/student/tasks')} className="p-6 bg-white dark:bg-
gray-800 border border-gray-100 dark:border-gray-700 rounded-3xl hover:shadow-xl transition-
all text-left flex items-start gap-4">
            <div className="p-3 bg-indigo-50 dark:bg-indigo-900/30 text-indigo-600 rounded-
2xl">
              <Layout size={24} />
            </div>
            <div>
              <h3 className="text-sm font-black">Open Task Board</h3>
              <p className="text-xs text-gray-500 mt-1">Manage sprints and kanban tasks</p>
            </div>
          </button>
          <button onClick={() => navigate('/student/github')} className="p-6 bg-white
```

```
dark:bg-gray-800 border border-gray-100 dark:border-gray-700 rounded-3xl hover:shadow-xl
transition-all text-left flex items-start gap-4">
              <div className="p-3 bg-gray-100 dark:bg-gray-700 text-gray-800 dark:text-gray-
200 rounded-2xl">
                <Github size={24} />
              </div>
              <div>
                <h3 className="text-sm font-black">Git Analytics</h3>
                <p className="text-xs text-gray-500 mt-1">View commit history and sync
repo</p>
              </div>
            </button>
          </div>
        </div>

        {/* Right: Entities & Details */}
        <div className="space-y-6">
          <Card>
            <h3 className="text-xs font-black uppercase tracking-widest text-gray-400 mb-
6">Group Composition</h3>
            <div className="space-y-4">
              {project.members.map((m, i) => (
                <div key={m} className="flex items-center gap-3">
                  <div className="w-10 h-10 bg-gray-100 dark:bg-gray-800 rounded-xl flex
items-center justify-center font-black text-xs text-blue-600">
                    {m.charAt(0)}
                  </div>
                  <div>
                    <p className="text-xs font-black">{m}</p>
                    {i === 0 && <span className="text-[8px] font-black uppercase text-blue-500
bg-blue-50 px-1 py-0.5 rounded">Group Lead</span>}
                  </div>
                </div>
              ))}
```

```
            </div>
            <button onClick={() => navigate('/student/chat')} className="w-full mt-6 py-3 bg-blue-50 dark:bg-blue-900/20 text-blue-600 dark:text-blue-400 rounded-2xl text-xs font-black flex items-center justify-center gap-2 hover:bg-blue-600 hover:text-white transition-all">
              <MessageSquare size={16} /> Group Messaging
            </button>
          </Card>

          <Card className="bg-gray-900 text-white">
            <h3 className="text-[10px] font-black text-gray-400 uppercase tracking-widest mb-4">Guide Detail</h3>
            <div className="flex items-center gap-3">
              <div className="w-10 h-10 bg-blue-600 rounded-xl flex items-center justify-center font-black text-xs">
                SJ
              </div>
              <div>
                <p className="text-sm font-black">{project.guide}</p>
                <p className="text-[10px] text-blue-400 font-bold">Assigned Mentor</p>
              </div>
            </div>
            <div className="mt-6 pt-6 border-t border-white/10 flex items-center justify-between">
              <div>
                <p className="text-[9px] font-black text-gray-400 uppercase">Batch Support</p>
                <p className="text-xs font-bold mt-1 uppercase">{project.batch}</p>
              </div>
              <button className="text-blue-400 hover:text-white transition-colors">
                <ArrowRight size={20} />
              </button>
            </div>
          </Card>
        </div>
```

```
      </div>
    </div>
  );
};

export default StudentDashboard;
```